

```
"""
```

```
Orca Sensory Cortex
```

```
=====
```

```
Translates raw OHLCV market data into physiological perceptions.
```

This is the missing link between The Genome (brain) and The Market (environment). Without this, the Orca hunts blind. With it, every state variable is grounded in real market conditions – and the sleep/consolidation/learning loop finally works.

```
Architecture:
```

```
Raw OHLCV → Technical Indicators → Physiological Mappings → OrganismState mutat
```

The key insight: don't just compute signal strength. Mutate the organism's BODY.

- VIX spike doesn't "reduce position size" → it RAISES THREAT LEVEL
- Volume surge doesn't "add to signal score" → it RAISES HUNTING ENERGY
- RSI overextension doesn't "block entry" → it INCREASES FATIGUE

That's what makes constraints form correctly during sleep consolidation.

```
"""
```

```
import pandas as pd
```

```
import numpy as np
```

```
from typing import Dict, List, Optional, Tuple
```

```
from dataclasses import dataclass
```

```
try:
```

```
    import pandas_ta as ta
```

```
    HAS_PANDAS_TA = True
```

```
except ImportError:
```

```
    HAS_PANDAS_TA = False
```

```
@dataclass
```

```
class SensoryPerception:
```

```
    """
```

```
    Multi-dimensional market perception.
```

```
    Each dimension maps to a physiological variable.
```

```
    """
```

```
    # What the organism SEES
```

```
    opportunity_score: float = 0.0
```

```
    # Prey visibility (feeds HUNTING_ENERGY +
```

```
    threat_score: float = 0.0
```

```
    # Environmental danger (feeds THREAT_LEVE
```

```
    exhaustion_score: float = 0.0
```

```
    # Trend fatigue (feeds FATIGUE, suppress
```

```
    regime_clarity: float = 0.5
```

```
    # How clear the current regime is (feeds
```

```

# Derived signal quality
signal_strength: float = 0.0          # Overall conviction for this setup
signal_direction: str = 'neutral'     # 'long', 'short', 'neutral'

# Context tags for constraint learning
context_tags: List[str] = None        # e.g. ['high_volatility', 'bull_regime',

def __post_init__(self):
    if self.context_tags is None:
        self.context_tags = []

def to_dict(self) -> Dict:
    return {
        'opportunity_score': self.opportunity_score,
        'threat_score': self.threat_score,
        'exhaustion_score': self.exhaustion_score,
        'regime_clarity': self.regime_clarity,
        'signal_strength': self.signal_strength,
        'signal_direction': self.signal_direction,
        'context_tags': self.context_tags
    }

class SensoryCortex:
    """
    The Orca's sensory system.

    Computes technical indicators from OHLCV data and maps them
    directly to physiological state mutations and signal perceptions.

    Works with daily OHLCV data (historical_data.pkl format).
    Maintains a rolling window per symbol for indicator computation.
    """

    def __init__(self, lookback: int = 50):
        self.lookback = lookback
        self.price_history: Dict[str, List[Dict]] = {} # symbol → OHLCV history
        self.market_history: List[Dict] = []           # SPY/VIX history

# -----
# PUBLIC API
# -----

def ingest_market_day(self, market_data: Dict):
    """
    Ingest a day's market data into the rolling history.
    Call this at the start of each cycle before perceive().

```

```

"""
# Update macro history
self.market_history.append({
    'spy_price': market_data.get('spy_price', 450),
    'spy_change_pct': market_data.get('spy_change_pct', 0),
    'vix': market_data.get('vix', 15),
    'vix_change_pct': market_data.get('vix_change_pct', 0),
})

if len(self.market_history) > self.lookback:
    self.market_history = self.market_history[-self.lookback:]

# Update per-symbol price history
all_prices = market_data.get('all_prices', {})
for symbol, price_data in all_prices.items():
    if symbol not in self.price_history:
        self.price_history[symbol] = []

    if isinstance(price_data, dict):
        self.price_history[symbol].append({
            'close': price_data.get('close', 0),
            'high': price_data.get('high', price_data.get('close', 0)),
            'low': price_data.get('low', price_data.get('close', 0)),
            'open': price_data.get('open', price_data.get('close', 0)),
            'volume': price_data.get('volume', 1_000_000),
        })
    else:
        # Direct float
        self.price_history[symbol].append({
            'close': float(price_data), 'high': float(price_data),
            'low': float(price_data), 'open': float(price_data),
            'volume': 1_000_000
        })

    if len(self.price_history[symbol]) > self.lookback:
        self.price_history[symbol] = self.price_history[symbol][-self.lookback:]

def perceive_market(self, market_data: Dict, state) -> Tuple[Dict, List[Dict]]
"""

Main perception pipeline.

Returns:
    (macro_perception, watchlist_signals)

macro_perception: Dict with regime, vix stats, mutates state physiology
watchlist_signals: List of {symbol, perception: SensoryPerception, ...}
"""

# 1. Macro environment perception (mutates threat level, regime)

```

```

macro = self._perceive_macro(market_data, state)

# 2. Per-symbol opportunity detection
signals = []
all_prices = market_data.get('all_prices', {})

for symbol in all_prices:
    if symbol == 'SPY':
        continue
    perception = self._perceive_symbol(symbol, macro)
    if perception.signal_strength > 0.25: # minimum threshold
        signals.append({
            'symbol': symbol,
            'ticker': symbol,
            'price': self._last_price(symbol),
            'strength': perception.signal_strength,
            'direction': perception.signal_direction,
            'setup_type': self._classify_setup(perception),
            'volume': self._last_volume(symbol),
            'perception': perception,
            # Breakout flag for legacy orca.py compatibility
            'breakout': perception.opportunity_score > 0.6,
            'rsi': self._last_rsi(symbol),
            'volume_ratio': self._volume_ratio(symbol),
            'regime': macro.get('regime', 'unknown'),
        })

# Sort by signal strength descending
signals.sort(key=lambda x: x['strength'], reverse=True)

return macro, signals

def mutate_state_from_macro(self, state, macro: Dict):
    """
    Apply macro environmental perception to organism physiology.

    This is the KEY step: market conditions physically change the organism's
    Not just signal scores – actual state variable mutations.
    """
    vix = macro.get('vix', 15)
    vix_norm = macro.get('vix_normalized', 0.3)
    regime = macro.get('regime', 'unknown')
    vix_spike = macro.get('vix_spike', False)

    # VIX → THREAT LEVEL (organism feels unsafe in volatile markets)
    threat_delta = 0.0
    if vix > 30:

```

```

        threat_delta = 0.3      # Crisis: major threat surge
    elif vix > 25:
        threat_delta = 0.15     # High vol: notable threat
    elif vix > 20:
        threat_delta = 0.05     # Elevated vol: mild threat
    else:
        threat_delta = -0.05    # Calm market: threat decays

    if vix_spike:
        threat_delta += 0.2     # Sudden VIX jump = additional fear

    state.fear_level = max(0.0, min(1.0, state.fear_level + threat_delta))

    # Regime → CONFIDENCE modulation
    if regime == 'bull' and macro.get('spy_trend', 0) > 0:
        state.confidence = min(1.0, state.confidence + 0.02)
    elif regime in ('bear', 'crisis'):
        state.confidence = max(0.0, state.confidence - 0.03)
    elif regime == 'choppy':
        state.confidence = max(0.1, state.confidence - 0.01)

    # VIX → PATIENCE (scared organisms wait longer before striking)
    if vix > 25:
        state.patience_level = min(1.0, state.patience_level + 0.1)

    # Store regime in state
    state.market_regime = regime

    # Volatility regime tag
    if vix > 30:
        state.volatility_regime = 'extreme'
    elif vix > 22:
        state.volatility_regime = 'high'
    elif vix < 14:
        state.volatility_regime = 'low'
    else:
        state.volatility_regime = 'normal'

def build_physiological_context(self, symbol: str, state, macro: Dict) -> Dict
    """
    Build the full context dict for memory logging.
    Includes both market state AND organism state – so constraints
    can reference actual market conditions during sleep consolidation.
    """
    perception = self._perceive_symbol(symbol, macro)
    return {
        # Market context (what the organism perceived)

```

```

        'market_regime': macro.get('regime', 'unknown'),
        'volatility_regime': macro.get('volatility_regime', 'normal'),
        'vix': macro.get('vix', 15),
        'threat_at_entry': macro.get('vix_normalized', 0.3),
        'opportunity_score': perception.opportunity_score,
        'exhaustion_score': perception.exhaustion_score,
        'signal_strength': perception.signal_strength,
        'context_tags': perception.context_tags,

        # Organism physiology at time of decision
        'mode': getattr(state, 'market_regime', 'unknown'),
        'prey_density': state.prey_density,
        'confidence': state.confidence,
        'injury_level': state.injury_level,
        'fear_level': state.fear_level,
        'hunting_energy': state.hunting_energy,
        'portfolio_value': state.portfolio_value,
        'starving': state.is_starving(),
    }

# -----
# MACRO PERCEPTION
# -----

def _perceive_macro(self, market_data: Dict, state) -> Dict:
    """Compute macro-level market perception"""
    vix = market_data.get('vix', 15)
    spy_change = market_data.get('spy_change_pct', 0)
    vix_change = market_data.get('vix_change_pct', 0)

    # VIX normalized (0 = calm, 1 = panic)
    vix_normalized = min(1.0, max(0.0, (vix - 10) / 40))

    # VIX spike detection (sudden fear event)
    vix_spike = vix_change > 15 # VIX jumped 15%+ in a day

    # Regime detection using multi-factor approach
    regime = self._detect_regime_advanced(market_data)

    # SPY trend from history
    spy_trend = self._compute_spy_trend()

    # Prey density: how many symbols have valid setups
    # This gets computed in perceive_market() but we set a base here
    prey_density = 0.5 # placeholder, updated after signal scan

    # Predator competition proxy: high VIX + high volume = algos are active

```

```

    predator_competition = min(1.0, vix_normalized * 0.5 + abs(spy_change) *

return {
    'vix': vix,
    'vix_normalized': vix_normalized,
    'vix_spike': vix_spike,
    'spy_change': spy_change,
    'spy_trend': spy_trend,
    'regime': regime,
    'volatility_regime': 'extreme' if vix > 30 else 'high' if vix > 22 el
    'predator_competition': predator_competition,
    'regime_shift_detected': vix_spike or (abs(spy_change) > 2.0),
    'volatility_percentile': vix_normalized,
}

def _detect_regime_advanced(self, market_data: Dict) -> str:
    """Multi-factor regime detection"""
    vix = market_data.get('vix', 15)
    spy_change = market_data.get('spy_change_pct', 0)

    # SPY trend from history
    spy_trend = self._compute_spy_trend()

    if vix > 30:
        return 'crisis'
    elif vix > 25 and spy_change < -1.0:
        return 'bear'
    elif spy_trend > 0.01 and vix < 20:
        return 'bull'
    elif spy_trend < -0.01:
        return 'bear'
    elif abs(spy_change) < 0.3 and vix < 18:
        return 'choppy'
    else:
        return 'neutral'

def _compute_spy_trend(self) -> float:
    """Compute SPY short-term trend from history"""
    if len(self.market_history) < 5:
        return 0.0

    prices = [d['spy_price'] for d in self.market_history[-10:]]
    if len(prices) < 2:
        return 0.0

    # Simple linear regression slope normalized
    n = len(prices)

```

```

x = list(range(n))
mean_x = sum(x) / n
mean_y = sum(prices) / n

numerator = sum((x[i] - mean_x) * (prices[i] - mean_y) for i in range(n))
denominator = sum((x[i] - mean_x) ** 2 for i in range(n))

if denominator == 0:
    return 0.0

slope = numerator / denominator
# Normalize by price level
return slope / mean_y if mean_y > 0 else 0.0

# -----
# SYMBOL-LEVEL PERCEPTION
# -----

def _perceive_symbol(self, symbol: str, macro: Dict) -> SensoryPerception:
    """
    Compute multi-dimensional perception for a single symbol.
    Returns SensoryPerception with all physiological mappings.
    """
    history = self.price_history.get(symbol, [])

    if len(history) < 10:
        return SensoryPerception() # not enough data

    perception = SensoryPerception()
    context_tags = []

    # Build OHLCV arrays
    closes = [h['close'] for h in history]
    highs = [h['high'] for h in history]
    lows = [h['low'] for h in history]
    volumes = [h.get('volume', 1_000_000) for h in history]

    current_price = closes[-1]
    if current_price <= 0:
        return SensoryPerception()

    # --- INDICATOR COMPUTATION ---

    # RSI (momentum / exhaustion)
    rsi = self._compute_rsi(closes, 14)

    # ATR (volatility / threat)

```

```

atr = self._compute_atr(highs, lows, closes, 14)
atr_pct = atr / current_price if current_price > 0 else 0

# Volume ratio vs 20-day average
vol_ratio = self._compute_volume_ratio(volumes, 20)

# VWAP deviation (opportunity signal)
vwap_dev = self._compute_vwap_deviation(closes, highs, lows, volumes)

# Price momentum (5-day vs 20-day)
momentum_5 = self._compute_momentum(closes, 5)
momentum_20 = self._compute_momentum(closes, 20)

# Bollinger Band position
bb_pos = self._compute_bb_position(closes, 20, 2.0)

# Support/resistance breakout
is_breakout = self._detect_breakout(closes, highs, lows, 20)

# — PHYSIOLOGICAL MAPPINGS —————

# 1. OPPORTUNITY SCORE → feeds HUNTING_ENERGY and STRIKE drive
opportunity = 0.0

# Volume surge = scent of blood
if vol_ratio > 2.0:
    opportunity += 0.35
    context_tags.append('volume_surge')
elif vol_ratio > 1.5:
    opportunity += 0.20
    context_tags.append('volume_elevated')

# VWAP deviation = price dislocated from fair value (mean reversion prey)
if abs(vwap_dev) > 0.02:
    opportunity += 0.25
    if vwap_dev < -0.02:
        context_tags.append('below_vwap') # long opportunity
    else:
        context_tags.append('above_vwap') # short opportunity

# Breakout = directional prey
if is_breakout:
    opportunity += 0.30
    context_tags.append('breakout')

# Momentum alignment (5d and 20d pointing same direction)
if (momentum_5 > 0.01 and momentum_20 > 0.005) or (momentum_5 < -0.01 and

```

```

        opportunity += 0.10
        context_tags.append('momentum_aligned')

perception.opportunity_score = min(1.0, opportunity)

# 2. THREAT SCORE → feeds THREAT_LEVEL, suppresses STRIKE
threat = 0.0

# ATR % tells us how expensive movement is (cost of being wrong)
if atr_pct > 0.04:
    threat += 0.4
    context_tags.append('high_atr')
elif atr_pct > 0.025:
    threat += 0.2

# Macro VIX amplifies individual threat
vix_norm = macro.get('vix_normalized', 0.3)
threat += vix_norm * 0.4

# Crisis regime = maximum threat
if macro.get('regime') == 'crisis':
    threat += 0.3
    context_tags.append('crisis_regime')
elif macro.get('regime') == 'bear':
    threat += 0.15
    context_tags.append('bear_regime')

# VIX spike = sudden danger
if macro.get('vix_spike', False):
    threat += 0.2
    context_tags.append('vix_spike')

perception.threat_score = min(1.0, threat)

# 3. EXHAUSTION SCORE → feeds FATIGUE, suppresses ENTRY
exhaustion = 0.0

# Overbought/oversold RSI
if rsi > 75:
    exhaustion += 0.5
    context_tags.append('overbought')
elif rsi > 65:
    exhaustion += 0.2
elif rsi < 25:
    exhaustion += 0.4
    context_tags.append('oversold')
elif rsi < 35:

```

```

        exhaustion += 0.15

# Bollinger Band extremes
if bb_pos > 0.95 or bb_pos < 0.05:
    exhaustion += 0.3
    context_tags.append('bb_extreme')

# Momentum exhaustion (fast > slow by too much)
if abs(momentum_5) > abs(momentum_20) * 3:
    exhaustion += 0.2
    context_tags.append('momentum_exhausted')

perception.exhaustion_score = min(1.0, exhaustion)

# 4. REGIME CLARITY → feeds CONFIDENCE
clarity = 0.5
regime = macro.get('regime', 'unknown')
if regime in ('bull', 'bear'):
    clarity = 0.8 # Clear directional regime
elif regime == 'crisis':
    clarity = 0.9 # Crisis is very clear (just dangerous)
elif regime == 'choppy':
    clarity = 0.2 # Unclear = low confidence

perception.regime_clarity = clarity

# — SIGNAL DIRECTION —————

long_score = 0.0
short_score = 0.0

# Long signals
if vwap_dev < -0.015: long_score += 0.3
if rsi < 35: long_score += 0.3
if momentum_5 > 0 and momentum_20 > 0: long_score += 0.2
if bb_pos < 0.2: long_score += 0.2
if is_breakout and momentum_5 > 0: long_score += 0.2
if vol_ratio > 1.5 and momentum_5 > 0: long_score += 0.1

# Short signals
if vwap_dev > 0.015: short_score += 0.3
if rsi > 65: short_score += 0.3
if momentum_5 < 0 and momentum_20 < 0: short_score += 0.2
if bb_pos > 0.8: short_score += 0.2
if is_breakout and momentum_5 < 0: short_score += 0.2
if vol_ratio > 1.5 and momentum_5 < 0: short_score += 0.1

```

```

    if long_score > short_score and long_score > 0.3:
        perception.signal_direction = 'long'
    elif short_score > long_score and short_score > 0.3:
        perception.signal_direction = 'short'
    else:
        perception.signal_direction = 'neutral'

# — COMPOSITE SIGNAL STRENGTH —————
# High opportunity + low threat + low exhaustion = strong signal
# Gemini's insight: threat and exhaustion SUPPRESS strength, not just sco

raw_strength = perception.opportunity_score

# Threat penalty (dangerous environment reduces conviction)
threat_penalty = perception.threat_score * 0.4

# Exhaustion penalty (overextended reduces R/R)
exhaustion_penalty = perception.exhaustion_score * 0.3

# Regime bonus (clear regime improves conviction)
regime_bonus = (perception.regime_clarity - 0.5) * 0.2

# Direction requirement: neutral = no trade
direction_multiplier = 0.0 if perception.signal_direction == 'neutral' el

strength = (raw_strength - threat_penalty - exhaustion_penalty + regime_b
perception.signal_strength = max(0.0, min(1.0, strength))

# Add regime tag
if regime not in ('unknown',):
    context_tags.append(f'regime_{regime}')

perception.context_tags = context_tags

return perception

# —————
# TECHNICAL INDICATORS (pure Python, no ta-lib dependency)
# —————

def _compute_rsi(self, closes: List[float], period: int = 14) -> float:
    """Compute RSI"""
    if len(closes) < period + 1:
        return 50.0

    deltas = [closes[i] - closes[i-1] for i in range(1, len(closes))]
    recent = deltas[-period:]

```

```

gains = [d for d in recent if d > 0]
losses = [abs(d) for d in recent if d < 0]

avg_gain = sum(gains) / period if gains else 0.0
avg_loss = sum(losses) / period if losses else 0.0

if avg_loss == 0:
    return 100.0

rs = avg_gain / avg_loss
return 100 - (100 / (1 + rs))

def _compute_atr(self, highs: List[float], lows: List[float], closes: List[float]) -> float:
    """Compute Average True Range"""
    if len(closes) < 2:
        return 0.0

    true_ranges = []
    for i in range(1, len(closes)):
        tr = max(
            highs[i] - lows[i],
            abs(highs[i] - closes[i-1]),
            abs(lows[i] - closes[i-1])
        )
        true_ranges.append(tr)

    if not true_ranges:
        return 0.0

    return sum(true_ranges[-period:]) / min(len(true_ranges), period)

def _compute_volume_ratio(self, volumes: List[float], period: int = 20) -> float:
    """Volume ratio: current vs rolling average"""
    if len(volumes) < 2:
        return 1.0

    current = volumes[-1]
    avg = sum(volumes[-period-1:-1]) / min(len(volumes)-1, period)

    if avg == 0:
        return 1.0

    return current / avg

def _compute_vwap_deviation(self, closes: List[float], highs: List[float],
                             lows: List[float], volumes: List[float],

```

```

        period: int = 20) -> float:
    """VWAP deviation: how far current price is from VWAP"""
    if len(closes) < 2:
        return 0.0

    # Use recent window
    n = min(len(closes), period)
    c = closes[-n:]
    h = highs[-n:]
    l = lows[-n:]
    v = volumes[-n:] if len(volumes) >= n else [1_000_000] * n

    # Typical price
    typical_prices = [(h[i] + l[i] + c[i]) / 3 for i in range(n)]

    vwap_num = sum(typical_prices[i] * v[i] for i in range(n))
    vwap_den = sum(v)

    if vwap_den == 0:
        return 0.0

    vwap = vwap_num / vwap_den
    current = closes[-1]

    return (current - vwap) / vwap if vwap > 0 else 0.0

def _compute_momentum(self, closes: List[float], period: int) -> float:
    """Price momentum over N periods (normalized)"""
    if len(closes) < period + 1:
        return 0.0

    start = closes[-period-1]
    end = closes[-1]

    if start == 0:
        return 0.0

    return (end - start) / start

def _compute_bb_position(self, closes: List[float], period: int = 20, std_mul
    """Bollinger Band position (0=lower band, 0.5=middle, 1=upper band)"""
    if len(closes) < period:
        return 0.5

    recent = closes[-period:]
    mean = sum(recent) / len(recent)
    variance = sum((x - mean) ** 2 for x in recent) / len(recent)

```

```

std = variance ** 0.5

if std == 0:
    return 0.5

upper = mean + std_mult * std
lower = mean - std_mult * std
current = closes[-1]

band_range = upper - lower
if band_range == 0:
    return 0.5

return max(0.0, min(1.0, (current - lower) / band_range))

def _detect_breakout(self, closes: List[float], highs: List[float],
                    lows: List[float], period: int = 20) -> bool:
    """Detect price breakout above recent high or below recent low"""
    if len(closes) < period + 1:
        return False

    recent_highs = highs[-period-1:-1]
    recent_lows = lows[-period-1:-1]

    if not recent_highs or not recent_lows:
        return False

    resistance = max(recent_highs)
    support = min(recent_lows)
    current = closes[-1]

    # Breakout: price exceeds recent range by 0.5%
    return (current > resistance * 1.005) or (current < support * 0.995)

def _classify_setup(self, perception: SensoryPerception) -> str:
    """Classify setup type for memory logging"""
    tags = perception.context_tags

    if 'breakout' in tags and 'volume_surge' in tags:
        return 'volume_breakout'
    elif 'below_vwap' in tags or 'above_vwap' in tags:
        return 'vwap_reversion'
    elif 'overbought' in tags or 'oversold' in tags:
        return 'rsi_extreme'
    elif 'momentum_aligned' in tags:
        return 'momentum_continuation'
    else:

```

```

        return 'composite'

# -----
# HELPERS
# -----

def _last_price(self, symbol: str) -> float:
    history = self.price_history.get(symbol, [])
    return history[-1]['close'] if history else 0.0

def _last_volume(self, symbol: str) -> float:
    history = self.price_history.get(symbol, [])
    return history[-1].get('volume', 1_000_000) if history else 1_000_000

def _last_rsi(self, symbol: str) -> float:
    history = self.price_history.get(symbol, [])
    closes = [h['close'] for h in history]
    return self._compute_rsi(closes, 14)

def _volume_ratio(self, symbol: str) -> float:
    history = self.price_history.get(symbol, [])
    volumes = [h.get('volume', 1_000_000) for h in history]
    return self._compute_volume_ratio(volumes, 20)

def get_stats(self) -> Dict:
    """Return cortex statistics"""
    return {
        'symbols_tracked': len(self.price_history),
        'market_history_days': len(self.market_history),
        'symbols': list(self.price_history.keys()),
    }

```